

SMB

Server Message Block (SMB) is a client-server protocol that regulates access to files and entire directories and other network resources such as printers, routers, or interfaces released for the network. Information exchange between different system processes can also be handled based on the SMB protocol. **SMB** first became available to a broader public, for example, as part of the OS/2 network operating system LAN Manager and LAN Server. Since then, the main application area of the protocol has been the Windows operating system series in particular, whose network services support SMB in a downward-compatible manner - which means that devices with newer editions can easily communicate with devices that have an older Microsoft operating system installed. With the free software project Samba, there is also a solution that enables the use of SMB in Linux and Unix distributions and thus cross-platform communication via SMB.

The SMB protocol enables the client to communicate with other participants in the same network to access files or services shared with it on the network. The other system must also have implemented the network protocol and received and processed the client request using an SMB server application. Before that, however, both parties must establish a connection, which is why they first exchange corresponding messages.

In IP networks, SMB uses TCP protocol for this purpose, which provides for a three-way handshake between client and server before a connection is finally established. The specifications of the TCP protocol also govern the subsequent transport of data. We can take a look at some examples [here](#).

An SMB server can provide arbitrary parts of its local file system as shares. Therefore the hierarchy visible to a client is partially independent of the structure on the server. Access rights are defined by **Access Control Lists (ACL)**. They can be controlled in a fine-grained manner based on attributes such as **execute**, **read**, and **full access** for individual users or user groups. The ACLs are defined based on the shares and therefore do not correspond to the rights assigned locally on the server.

Samba

As mentioned earlier, there is an alternative variant to the SMB server, called Samba, developed for Unix-based operating system. Samba implements the **Common Internet File System (CIFS)** network protocol. **CIFS** is a "dialect" of SMB. In other words, CIFS is a very specific implementation of the SMB protocol, which in turn was created by Microsoft. This allows Samba to communicate with newer Windows systems. Therefore, it usually is referred to as **SMB / CIFS**. However, CIFS is the extension of the SMB protocol. So when we pass SMB commands over Samba to an older NetBIOS service, it usually connects to the Samba server over TCP ports **137**, **138**, **139**, but CIFS uses TCP port **445** only. There are several versions of SMB, including outdated versions that are still used in specific infrastructures.

SMB Version	Supported	Features
CIFS	Windows NT 4.0	Communication via NetBIOS interface
SMB 1.0	Windows 2000	Direct connection via TCP
SMB 2.0	Windows Vista, Windows Server 2008	Performance upgrades, improved message signing, caching feature
SMB 2.1	Windows 7, Windows Server 2008 R2	Locking mechanisms
SMB 3.0	Windows 8, Windows Server 2012	Multichannel connections, end-to-end encryption, remote storage access
SMB 3.0.2	Windows 8.1, Windows Server 2012 R2	
SMB 3.1.1	Windows 10, Windows Server 2016	Integrity checking, AES-128 encryption

With version 3, the Samba server gained the ability to be a full member of an Active Directory domain. With version 4, Samba even provides an Active Directory domain controller. It contains several so-called daemons for this purpose - which are Unix background programs. The SMB server daemon (**smbd**) belonging to Samba provides the first two functionalities, while the NetBIOS message block daemon (**nmbd**) implements the last two functionalities. The SMB service controls these two background programs.

We know that Samba is suitable for both Linux and Windows systems. In a network, each host participates in the same **workgroup**. A workgroup is a group name that identifies an arbitrary collection of computers and their resources on an SMB network. There can be multiple workgroups on the network at any given time. IBM developed an **application programming interface (API)** for networking computers called the **Network Basic Input/Output System (NetBIOS)**. The NetBIOS API provided a blueprint for an application to connect and share data with other computers. In a NetBIOS environment, when a machine goes online, it needs a name, which is done through the so-called **name registration** procedure. Either each host reserves its hostname on the network, or the **NetBIOS Name Server (NBNS)** is used for this purpose. It also has been enhanced to **Windows Internet Name Service (WINS)**.

Default Configuration

As we can imagine, Samba offers a wide range of **settings** that we can configure. Again, we define the settings via a text file where we can get an overview of some of the settings. These settings look like the following when filtered out:

Default Configuration

Default Configuration

```
dbcyp0n@htb[/htb]$ cat /etc/samba/smb.conf | grep -v "#\\|\\;"

[global]
    workgroup = DEV.INFREIGHT.HTB
    server string = DEV.SMB
    log file = /var/log/samba/log.%m
    max log size = 1000
    logging = file
    panic action = /usr/share/samba/panic-action %d

    server role = standalone server
    obey pam restrictions = yes
    unix password sync = yes

    passwd program = /usr/bin/passwd %u
    passwd chat = *Enter\snew\s*\spassword:* %n\n *Retye\snew\s*\spassword:* %n\n *password\supdated\ssuccessfull

    pam password change = yes
    map to guest = bad user
    usershare allow guests = yes

[printers]
    comment = All Printers
    browseable = no
    path = /var/spool/samba
    printable = yes
    guest ok = no
    read only = yes
    create mask = 0700

[print$]
    comment = Printer Drivers
    path = /var/lib/samba/printers
    browseable = yes
    read only = yes
    guest ok = no
```

We see global settings and two shares that are intended for printers. The global settings are the configuration of the available SMB server that is used for all shares. In the individual shares, however, the global settings can be overwritten, which can be configured with high probability even incorrectly. Let us look at some of the settings to understand how the shares are configured in Samba.

Setting	Description
[sharename]	The name of the network share.
workgroup = WORKGROUP/DOMAIN	Workgroup that will appear when clients query.
path = /path/here/	The directory to which user is to be given access.
server string = STRING	The string that will show up when a connection is initiated.
unix password sync = yes	Synchronize the UNIX password with the SMB password?
usershare allow guests = yes	Allow non-authenticated users to access defined share?
map to guest = bad user	What to do when a user login request doesn't match a valid UNIX user?
browseable = yes	Should this share be shown in the list of available shares?
guest ok = yes	Allow connecting to the service without using a password?
read only = yes	Allow users to read files only?
create mask = 0700	What permissions need to be set for newly created files?

Dangerous Settings

Some of the above settings already bring some sensitive options. However, suppose we question the settings listed below and ask ourselves what the employees could gain from them, as well as attackers. In that case, we will see what advantages and disadvantages the settings bring with them. Let us take the setting `browseable = yes` as an example. If we as administrators adopt this setting, the company's employees will have the comfort of being able to look at the individual folders with the contents. Many folders are eventually used for better organization and structure. If the employee can browse through the shares, the attacker will also be able to do so after successful access.

Setting	Description
browseable = yes	Allow listing available shares in the current share?
read only = no	Forbid the creation and modification of files?
writable = yes	Allow users to create and modify files?
guest ok = yes	Allow connecting to the service without using a password?
enable privileges = yes	Honor privileges assigned to specific SID?
create mask = 0777	What permissions must be assigned to the newly created files?
directory mask = 0777	What permissions must be assigned to the newly created directories?
logon script = script.sh	What script needs to be executed on the user's login?
magic script = script.sh	Which script should be executed when the script gets closed?
magic output = script.out	Where the output of the magic script needs to be stored?

Let us create a share called [notes] and a few others and see how the settings affect our enumeration process. We will use all of the above settings and apply them to this share. For example, this setting is often applied, if only for testing purposes. If it is then an internal subnet of a small team in a large department, this setting is often retained or forgotten to be reset. This leads to the fact that we can browse through all the shares and, with high probability, even download and inspect them.

Example Share

Example Share

```
...SNIP...

[notes]
    comment = CheckIT
    path = /mnt/notes/

    browseable = yes
    read only = no
    writable = yes
    guest ok = yes

    enable privileges = yes
    create mask = 0777
    directory mask = 0777
```

It is highly recommended to look at the man pages for Samba and configure it ourselves and experiment with the settings. We will then discover potential aspects that will be interesting for us as a penetration tester. In addition, the more familiar we become with the Samba server and SMB, the easier it will be to find our way around the environment and use it for our purposes. Once we have adjusted /etc/samba/smb.conf to our needs, we have to restart the service on the server.

Restart Samba

Restart Samba

```
root@samba:~# sudo systemctl restart smbd
```

Now we can display a list (-L) of the server's shares with the smbclient command from our host. We use the so-called null session (-N), which is anonymous access without the input of existing users or valid passwords.

SMBclient - Connecting to the Share

SMBclient - Connecting to the Share

```
dbcypth0n@htb[/htb]$ smbclient -N -L //10.129.14.128

      Sharename      Type      Comment
      -----      -
      print$         Disk      Printer Drivers
      home           Disk      INFREIGHT Samba
      dev            Disk      DEVenv
      notes          Disk      CheckIT
      IPC$           IPC       IPC Service (DEVSM)
SMB1 disabled -- no workgroup available
```

We can see that we now have five different shares on the Samba server from the result. Thereby print\$ and an IPC\$ are already included by default in the basic setting, as we have already seen. Since we deal with the [notes] share, let us log in and inspect it using the same client program. If we are not familiar with the client program, we can use the help command on successful login, listing all the possible commands we can execute.

SMBclient - Connecting to the Share

```
dbcypth0n@htb[/htb]$ smbclient //10.129.14.128/notes
```

```
Enter WORKGROUP\<username>'s password:
Anonymous login successful
Try "help" to get a list of possible commands.
```

```
smb: \> help
```

?	allinfo	altname	archive	backup
blocksize	cancel	case_sensitive	cd	chmod
chown	close	del	deltree	dir
du	echo	exit	get	getfacl
geteas	hardlink	help	history	iosize
lcd	link	lock	lowercase	ls
l	mask	md	mget	mkdir
more	mput	newer	notify	open
posix	posix_encrypt	posix_open	posix_mkdir	posix_rmdir
posix_unlink	posix_whoami	print	prompt	put
pwd	q	queue	quit	readlink
rd	recurse	reget	rename	reput
rm	rmdir	showacls	setea	setmode
scopy	stat	symlink	tar	tarmode
timeout	translate	unlock	volume	vuid
wdel	logon	listconnect	showconnect	tcon
tdis	tid	utimes	logoff	..
!				

```
smb: \> ls
```

.	D	0	Wed Sep 22 18:17:51 2021
..	D	0	Wed Sep 22 12:03:59 2021
prep-prod.txt	N	71	Sun Sep 19 15:45:21 2021

```
30313412 blocks of size 1024. 16480084 blocks available
```

Once we have discovered interesting files or folders, we can download them using the `get` command. Smbclient also allows us to execute local system commands using an exclamation mark at the beginning (`!<cmd>`) without interrupting the connection.

Download Files from SMB

Download Files from SMB

```
smb: \> get prep-prod.txt
```

```
getting file \prep-prod.txt of size 71 as prep-prod.txt (8,7 KiloBytes/sec)
(average 8,7 KiloBytes/sec)
```

```
smb: \> !ls
```

```
prep-prod.txt
```

```
smb: \> !cat prep-prod.txt
```

```
[ ] check your code with the templates
[ ] run code-assessment.py
[ ] ...
```

From the administrative point of view, we can check these connections using `smbstatus`. Apart from the Samba version, we can also see who, from which host, and which share the client is connected. This is especially important once we have entered a subnet (perhaps even an isolated one) that the others can still access.

For example, with domain-level security, the samba server acts as a member of a Windows domain. Each domain has at least one domain controller, usually a Windows NT server providing password authentication. This domain controller provides the workgroup with a definitive password server. The domain controllers keep track of users and passwords in their own **Security Authentication Module (SAM)** and authenticate each user when they log in for the first time and wish to access another machine's share.

Samba Status

```
Samba Status

root@samba:~# smbstatus

Samba version 4.11.6-Ubuntu
PID      Username      Group        Machine
-----
75691    sambauser     samba        10.10.14.4 (ipv4:10.10.14.4:45564)
Protocol Version  Encryption
-----
SMB3_11      -

Service      pid      Machine      Connected at      Encryption      Signing
-----
notes         75691    10.10.14.4    Do Sep 23 00:12:06 2021 CEST      -              -

No locked files
```

Footprinting the Service

Let us go back to one of our enumeration tools. Nmap also has many options and NSE scripts that can help us examine the target's SMB service more closely and get more information. The downside, however, is that these scans can take a long time. Therefore, it is also recommended to look at the service manually, mainly because we can find much more details than Nmap could show us. First, however, let us see what Nmap can find on our target Samba server, where we created the **[notes]** share for testing purposes.

Nmap

```
Nmap

dbcypth0n@htb[/htb]$ sudo nmap 10.129.14.128 -sV -sC -p139,445

Starting Nmap 7.80 ( https://nmap.org ) at 2021-09-19 15:15 CEST
Nmap scan report for sharing.inlanefreight.htb (10.129.14.128)
Host is up (0.00024s latency).

PORT      STATE SERVICE      VERSION
139/tcp   open  netbios-ssn Samba smbd 4.6.2
445/tcp   open  netbios-ssn Samba smbd 4.6.2
MAC Address: 00:00:00:00:00:00 (VMware)

Host script results:
|_ nbstat: NetBIOS name: HTB, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
| smb2-security-mode:
|   2.02:
|_   Message signing enabled but not required
| smb2-time:
|   date: 2021-09-19T13:16:04
|_   start_date: N/A

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.35 seconds
```

We can see from the results that it is not very much that Nmap provided us with here. Therefore, we should resort to other tools that allow us to interact manually with the SMB and send specific requests for the information. One of the handy tools for this is **rpcclient**. This is a tool to perform MS-RPC functions.

The [Remote Procedure Call \(RPC\)](#) is a concept and, therefore, also a central tool to realize operational and work-sharing structures in networks and client-server architectures. The communication process via RPC includes passing parameters and the return of a function value.

RPCclient

RPCclient

```
dbcypth0n@htb[/htb]$ rpcclient -U "" 10.129.14.128

Enter WORKGROUP\'s password:
rpcclient $>
```

The `rpcclient` offers us many different requests with which we can execute specific functions on the SMB server to get information. A complete list of all these functions can be found on the [man page](#) of the `rpcclient`.

Query	Description
<code>srvinfo</code>	Server information.
<code>enumdomains</code>	Enumerate all domains that are deployed in the network.
<code>querydomaininfo</code>	Provides domain, server, and user information of deployed domains.
<code>netshareenumall</code>	Enumerates all available shares.
<code>netsharegetinfo <share></code>	Provides information about a specific share.
<code>enumdomusers</code>	Enumerates all domain users.
<code>queryuser <RID></code>	Provides information about a specific user.

RPCclient - Enumeration

RPCclient - Enumeration

```
rpcclient $> srvinfo
```

```
DEVSMB      Wk Sv PrQ Unx NT SNT DEVSM
platform_id  :      500
os version   :      6.1
server type   :      0x809a03
```

```
rpcclient $> enumdomains
```

```
name:[DEVSMB] idx:[0x0]
name:[Builtin] idx:[0x1]
```

```
rpcclient $> querydominfo
```

```
Domain:      DEVOPS
Server:      DEVSMB
Comment:     DEVSM
Total Users: 2
Total Groups: 0
Total Aliases: 0
Sequence No: 1632361158
Force Logoff: -1
Domain Server State: 0x1
Server Role:  ROLE_DOMAIN_PDC
Unknown 3:   0x1
```

```
rpcclient $> netshareenumall
```

```
netname: print$
  remark: Printer Drivers
  path:   C:\var\lib\samba\printers
  password:
netname: home
  remark: INFREIGHT Samba
  path:   C:\home\
  password:
netname: dev
  remark: DEVenv
  path:   C:\home\sambauser\dev\
  password:
netname: notes
  remark: CheckIT
  path:   C:\mnt\notes\
  password:
netname: IPC$
  remark: IPC Service (DEVSM)
  path:   C:\tmp
  password:
```

```
rpcclient $> netsharegetinfo notes
```

```
netname: notes
  remark: CheckIT
  path:   C:\mnt\notes\
  password:
  type:   0x0
  perms:  0
  max_uses:      -1
  num_uses:      1
revision: 1
type: 0x8004: SEC_DESC_DACL_PRESENT SEC_DESC_SELF_RELATIVE
DACL
  ACL      Num ACEs:      1      revision:      2
  ---
  ACE
    type: ACCESS_ALLOWED (0) flags: 0x00
    Specific bits: 0x1ff
```



```
Permissions: 0x101f01ff: Generic all access SYNCHRONIZE_ACCESS WRITE_OWNER_ACCESS WRITE_DAC_ACCESS
SID: S-1-1-0
```

These examples show us what information can be leaked to anonymous users. Once an **anonymous** user has access to a network service, it only takes one mistake to give them too many permissions or too much visibility to put the entire network at significant risk.

Most importantly, anonymous access to such services can also lead to the discovery of other users, who can be attacked with brute-forcing in the most aggressive case. Humans are more error-prone than properly configured computer processes, and the lack of security awareness and laziness often leads to weak passwords that can be easily cracked. Let us see how we can enumerate users using the **rpcclient**.

Rpcclient - User Enumeration

Rpcclient - User Enumeration

```

rpcclient $> enumdomusers

user:[mrb3n] rid:[0x3e8]
user:[cry0l1t3] rid:[0x3e9]

rpcclient $> queryuser 0x3e9

User Name      : cry0l1t3
Full Name      : cry0l1t3
Home Drive     : \\devsmb\cry0l1t3
Dir Drive      : 
Profile Path   : \\devsmb\cry0l1t3\profile
Logon Script:
Description    : 
Workstations:
Comment        : 
Remote Dial    : 
Logon Time     :      Do, 01 Jan 1970 01:00:00 CET
Logoff Time    :      Mi, 06 Feb 2036 16:06:39 CET
Kickoff Time   :      Mi, 06 Feb 2036 16:06:39 CET
Password last set Time : Mi, 22 Sep 2021 17:50:56 CEST
Password can change Time : Mi, 22 Sep 2021 17:50:56 CEST
Password must change Time: Do, 14 Sep 30828 04:48:05 CEST
unknown_2[0..31]...
user_rid      :      0x3e9
group_rid     :      0x201
acb_info      :      0x00000014
fields_present: 0x00ffffff
logon_divs    :      168
bad_password_count:      0x00000000
logon_count   :      0x00000000
padding1[0..7]...
logon_hrs[0..21]...

rpcclient $> queryuser 0x3e8

User Name      : mrb3n
Full Name      : 
Home Drive     : \\devsmb\mrb3n
Dir Drive      : 
Profile Path   : \\devsmb\mrb3n\profile
Logon Script:
Description    : 
Workstations:
Comment        : 
Remote Dial    : 
Logon Time     :      Do, 01 Jan 1970 01:00:00 CET
Logoff Time    :      Mi, 06 Feb 2036 16:06:39 CET
Kickoff Time   :      Mi, 06 Feb 2036 16:06:39 CET
Password last set Time : Mi, 22 Sep 2021 17:47:59 CEST
Password can change Time : Mi, 22 Sep 2021 17:47:59 CEST
Password must change Time: Do, 14 Sep 30828 04:48:05 CEST
unknown_2[0..31]...
user_rid      :      0x3e8
group_rid     :      0x201
acb_info      :      0x00000010
fields_present: 0x00ffffff
logon_divs    :      168
bad_password_count:      0x00000000
logon_count   :      0x00000000
padding1[0..7]...
logon_hrs[0..21]...

```

We can then use the results to identify the group's RID, which we can then use to retrieve information from the entire group.

Rpcclient - Group Information

```
rpcclient $> querygroup 0x201
```

```
Group Name:      None
Description:     Ordinary Users
Group Attribute: 7
Num Members: 2
```

However, it can also happen that not all commands are available to us, and we have certain restrictions based on the user. However, the query `queryuser <RID>` is mostly allowed based on the RID. So we can use the `rpcclient` to brute force the RIDs to get information. Because we may not know who has been assigned which RID, we know that we will get information about it as soon as we query an assigned RID. There are several ways and tools we can use for this. To stay with the tool, we can create a `For-loop` using `Bash` where we send a command to the service using `rpcclient` and filter out the results.

Brute Forcing User RIDs

Brute Forcing User RIDs

```
dbcyph0n@htb[/htb]$ for i in $(seq 500 1100);do rpcclient -N -U "" 10.129.14.128 -c "queryuser 0x$(printf '%x\n'
```

User Name	:	sambauser
user_rid	:	0x1f5
group_rid:		0x201
User Name	:	mr3n
user_rid	:	0x3e8
group_rid:		0x201
User Name	:	cry011t3
user_rid	:	0x3e9
group_rid:		0x201

An alternative to this would be a Python script from [Impacket](#) called `samrdump.py`.

Impacket - Samrdump.py

Impacket - Samrdump.py

```

dbcypth0n@htb[/htb]$ samrdump.py 10.129.14.128

Impacket v0.9.22 - Copyright 2020 SecureAuth Corporation

[*] Retrieving endpoint list from 10.129.14.128
Found domain(s):
. DEVSMB
. Builttin
[*] Looking up users in domain DEVSMB
Found user: mrb3n, uid = 1000
Found user: cry0l1t3, uid = 1001
mrb3n (1000)/FullName:
mrb3n (1000)/UserComment:
mrb3n (1000)/PrimaryGroupId: 513
mrb3n (1000)/BadPasswordCount: 0
mrb3n (1000)/LogonCount: 0
mrb3n (1000)/PasswordLastSet: 2021-09-22 17:47:59
mrb3n (1000)/PasswordDoesNotExpire: False
mrb3n (1000)/AccountIsDisabled: False
mrb3n (1000)/ScriptPath:
cry0l1t3 (1001)/FullName: cry0l1t3
cry0l1t3 (1001)/UserComment:
cry0l1t3 (1001)/PrimaryGroupId: 513
cry0l1t3 (1001)/BadPasswordCount: 0
cry0l1t3 (1001)/LogonCount: 0
cry0l1t3 (1001)/PasswordLastSet: 2021-09-22 17:50:56
cry0l1t3 (1001)/PasswordDoesNotExpire: False
cry0l1t3 (1001)/AccountIsDisabled: False
cry0l1t3 (1001)/ScriptPath:
[*] Received 2 entries.

```

The information we have already obtained with `rpcclient` can also be obtained using other tools. For example, the `SMBMap` and `CrackMapExec` tools are also widely used and helpful for the enumeration of SMB services.

SMBmap

SMBmap

```

dbcypth0n@htb[/htb]$ smbmap -H 10.129.14.128

[+] Finding open SMB ports....
[+] User SMB session established on 10.129.14.128...
[+] IP: 10.129.14.128:445      Name: 10.129.14.128

```

Disk	Permissions	Comment
----	-----	-----
print\$	NO ACCESS	Printer Drivers
home	NO ACCESS	INFREIGHT Samba
dev	NO ACCESS	DEVenv
notes	NO ACCESS	CheckIT
IPC\$	NO ACCESS	IPC Service (DEVSM)

CrackMapExec

CrackMapExec

```
dbcyp0n@htb[/htb]$ crackmapexec smb 10.129.14.128 --shares -u '' -p ''

SMB      10.129.14.128  445    DEV SMB      [*] Windows 6.1 Build 0 (name:DEV SMB) (domain:) (signing:False)
SMB      10.129.14.128  445    DEV SMB      [+] \:
SMB      10.129.14.128  445    DEV SMB      [+] Enumerated shares
SMB      10.129.14.128  445    DEV SMB      Share      Permissions      Remark
SMB      10.129.14.128  445    DEV SMB      -----      -
SMB      10.129.14.128  445    DEV SMB      print$      Printer Drivers
SMB      10.129.14.128  445    DEV SMB      home      INFREIGHT Samba
SMB      10.129.14.128  445    DEV SMB      dev      DEVenv
SMB      10.129.14.128  445    DEV SMB      notes      READ,WRITE      CheckIT
SMB      10.129.14.128  445    DEV SMB      IPC$      IPC Service (DEVSM)
```

Another tool worth mentioning is the so-called [enum4linux-ng](#), which is based on an older tool, enum4linux. This tool automates many of the queries, but not all, and can return a large amount of information.

Enum4Linux-ng - Installation

Enum4Linux-ng - Installation

```
dbcyp0n@htb[/htb]$ git clone https://github.com/cddmp/enum4linux-ng.git
dbcyp0n@htb[/htb]$ cd enum4linux-ng
dbcyp0n@htb[/htb]$ pip3 install -r requirements.txt
```

Enum4Linux-ng - Enumeration

Enum4Linux-ng - Enumeration

```
dbcyp0n@htb[/htb]$ ./enum4linux-ng.py 10.129.14.128 -A
```

```
ENUM4LINUX - next generation
```

```
=====
|   Target Information   |
=====
[*] Target ..... 10.129.14.128
[*] Username ..... ''
[*] Random Username .. 'juzgtcsu'
[*] Password ..... ''
[*] Timeout ..... 5 second(s)

=====
|   Service Scan on 10.129.14.128   |
=====
[*] Checking LDAP
[-] Could not connect to LDAP on 389/tcp: connection refused
[*] Checking LDAPS
[-] Could not connect to LDAPS on 636/tcp: connection refused
[*] Checking SMB
[+] SMB is accessible on 445/tcp
[*] Checking SMB over NetBIOS
[+] SMB over NetBIOS is accessible on 139/tcp

=====
|   NetBIOS Names and Workgroup for 10.129.14.128   |
=====
[+] Got domain/workgroup name: DEVOPS
[+] Full NetBIOS names information:
- DEVSMB      <00> -      H <ACTIVE> Workstation Service
- DEVSMB      <03> -      H <ACTIVE> Messenger Service
- DEVSMB      <20> -      H <ACTIVE> File Server Service
- .._MSBROWSE_. <01> - <GROUP> H <ACTIVE> Master Browser
- DEVOPS      <00> - <GROUP> H <ACTIVE> Domain/Workgroup Name
- DEVOPS      <1d> -      H <ACTIVE> Master Browser
- DEVOPS      <1e> - <GROUP> H <ACTIVE> Browser Service Elections
- MAC Address = 00-00-00-00-00-00

=====
|   SMB Dialect Check on 10.129.14.128   |
=====
[*] Trying on 445/tcp
[+] Supported dialects and settings:
SMB 1.0: false
SMB 2.02: true
SMB 2.1: true
SMB 3.0: true
SMB1 only: false
Preferred dialect: SMB 3.0
SMB signing required: false

=====
|   RPC Session Check on 10.129.14.128   |
=====
[*] Check for null session
[+] Server allows session using username '', password ''
[*] Check for random user session
[+] Server allows session using username 'juzgtcsu', password ''
[H] Rerunning enumeration with user 'juzgtcsu' might give more results

=====
|   Domain Information via RPC for 10.129.14.128   |
=====
[+] Domain: DEVOPS
[+] SID: NULL SID
[+] Host is part of a workgroup (not a domain)

=====
|   Domain Information via SMB session for 10.129.14.128   |
=====
[*] Enumerating via unauthenticated SMB session on 445/tcp
[+] Found domain information via SMB
NetBIOS computer name: DEVSMB
```

NetBIOS domain name: ''
DNS domain: ''
FQDN: htb

```
=====
|   OS Information via RPC for 10.129.14.128   |
=====
[*] Enumerating via unauthenticated SMB session on 445/tcp
[+] Found OS information via SMB
[*] Enumerating via 'srvinfo'
[+] Found OS information via 'srvinfo'
[+] After merging OS information we have the following result:
OS: Windows 7, Windows Server 2008 R2
OS version: '6.1'
OS release: ''
OS build: '0'
Native OS: not supported
Native LAN manager: not supported
Platform id: '500'
Server type: '0x809a03'
Server type string: Wk Sv PrQ Unx NT SNT DEVSM
```

```
=====
|   Users via RPC on 10.129.14.128   |
=====
[*] Enumerating users via 'querydispinfo'
[+] Found 2 users via 'querydispinfo'
[*] Enumerating users via 'enumdomusers'
[+] Found 2 users via 'enumdomusers'
[+] After merging user results we have 2 users total:
'1000':
  username: mrb3n
  name: ''
  acb: '0x00000010'
  description: ''
'1001':
  username: cry011t3
  name: cry011t3
  acb: '0x00000014'
  description: ''
```

```
=====
|   Groups via RPC on 10.129.14.128   |
=====
[*] Enumerating local groups
[+] Found 0 group(s) via 'enumalsgroups domain'
[*] Enumerating builtin groups
[+] Found 0 group(s) via 'enumalsgroups builtin'
[*] Enumerating domain groups
[+] Found 0 group(s) via 'enumdomgroups'
```

```
=====
|   Shares via RPC on 10.129.14.128   |
=====
[*] Enumerating shares
[+] Found 5 share(s):
IPC$:
  comment: IPC Service (DEVSM)
  type: IPC
dev:
  comment: DEVenv
  type: Disk
home:
  comment: INFREIGHT Samba
  type: Disk
notes:
  comment: CheckIT
  type: Disk
print$:
  comment: Printer Drivers
  type: Disk
[*] Testing share IPC$
[-] Could not check share: STATUS_OBJECT_NAME_NOT_FOUND
[*] Testing share dev
[-] Share doesn't exist
```

```
[*] Testing share home
[+] Mapping: OK, Listing: OK
[*] Testing share notes
[+] Mapping: OK, Listing: OK
[*] Testing share print$
[+] Mapping: DENIED, Listing: N/A

=====
| Policies via RPC for 10.129.14.128 |
=====
[*] Trying port 445/tcp
[+] Found policy:
domain_password_information:
  pw_history_length: None
  min_pw_length: 5
  min_pw_age: none
  max_pw_age: 49710 days 6 hours 21 minutes
  pw_properties:
    - DOMAIN_PASSWORD_COMPLEX: false
    - DOMAIN_PASSWORD_NO_ANON_CHANGE: false
    - DOMAIN_PASSWORD_NO_CLEAR_CHANGE: false
    - DOMAIN_PASSWORD_LOCKOUT_ADMINS: false
    - DOMAIN_PASSWORD_PASSWORD_STORE_CLEARTEXT: false
    - DOMAIN_PASSWORD_REFUSE_PASSWORD_CHANGE: false
domain_lockout_information:
  lockout_observation_window: 30 minutes
  lockout_duration: 30 minutes
  lockout_threshold: None
domain_logoff_information:
  force_logoff_time: 49710 days 6 hours 21 minutes

=====
| Printers via RPC for 10.129.14.128 |
=====
[+] No printers returned (this is not an error)

Completed after 0.61 seconds
```

We need to use more than two tools for enumeration. Because it can happen that due to the programming of the tools, we get different information that we have to check manually. Therefore, we should never rely only on automated tools where we do not know precisely how they were written.

VPN Servers

 Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE

Start Instance

∞ / 1 spawns left

Waiting to start..

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: 10.129.202.5 

Life Left: 23 minutes 

 Cheat Sheet

 Download VPN Connection File

+0  What version of the SMB server is running on the target system? Submit the entire banner as the answer.

Submit your answer here...

 Submit

+0  What is the name of the accessible share on the target?

Submit your answer here...

 Submit

+0  Connect to the discovered share and find the flag.txt file. Submit the contents as the answer.

Submit your answer here...

 Submit

+0  Find out which domain the server belongs to.

Submit your answer here...

 Submit

 Hint

+ 0 

 Find additional information about the specific share we found previously and submit the customized version of that specific share as the answer.

Submit your answer here...

 Submit

+ 0 

 What is the full system path of that specific share?

Submit your answer here...

 Submit

 Hint

 Previous

Next 

 Cheat Sheet

 Resources

 Go to Questions

Table of Contents

Introduction

Enumeration Principles	
Enumeration Methodology	

Infrastructure Based Enumeration

Domain Information	
Cloud Resources	
Staff	

Host Based Enumeration

 FTP	
 SMB	
 NFS	
 DNS	
 SMTP	
 IMAP / POP3	
 SNMP	
 MySQL	

 MSSQL

 Oracle TNS

 IPMI

Remote Management Protocols

Linux Remote Management Protocols

Windows Remote Management Protocols

Skills Assessment

 Footprinting Lab - Easy

 Footprinting Lab - Medium

 Footprinting Lab - Hard

My Workstation

OFFLINE

 Start Instance

 / 1 spawns left